Constructor :

1. What is a constructor in Java? How is it different from a method?
2. Can a constructor be final, abstract, or static? Why or why not?
3. What is a default constructor? When is it provided by the compiler?
4. Can a constructor call another constructor in the same class? (Explain this())
5. What is constructor overloading? Give an example.
6. Can a constructor return a value? If not, why?
7. Can we call a constructor explicitly like a method?
8. What happens if you define a parameterized constructor but not a default one?
9. Can a constructor throw exceptions?
10. Difference between **constructor chaining** using this() and super().

Method:

1. Difference between **method overloading** and **method overriding**.
2. What are the rules for method overloading in Java?
3. What are the rules for method overriding? (Access modifiers, return types, exceptions)
4. What is **covariant return type** in method overriding?
5. Can we override a static method? Why or why not?
6. Can we override a private or final method?
7. What is the difference between **compile-time polymorphism** and **runtime polymorphism**?
8. Explain **method hiding** with an example.
9. Can two methods have the same name if they differ only in return type?
10. Difference between equals() and == method.

Instance (Objects) :

1. What is the difference between an **instance variable** and a **local variable**?
2. When are instance variables initialized?
3. How are instance blocks different from static blocks?
4. Can instance variables be declared as static?

5. What is the difference between **instance methods** and **static methods**?
6. How do you access instance variables in a static context?
7. What happens if you try to use an uninitialized instance variable?
8. Difference between **shallow copy** and **deep copy** of an object.
9. How is memory allocated for objects (heap vs stack)?
10. What happens if you don't explicitly create an object in Java?

Static:

1. What is the difference between a **static variable** and an **instance variable**?
2. What is a **static method**? Why can't it access instance variables directly?
3. What are **static blocks**? When are they executed?
4. Can we have multiple static blocks in a class? If yes, in which order do they execute?
5. Can we override static methods? Why or why not?
6. What is a **static nested class**? How is it different from a non-static inner class?
7. Can a constructor be static? Why not?
8. When do static variables get initialized? (Class loading vs object creation)
9. Can we use this or super inside a static method? Why not?
10. Give a real-world example where static variables are useful.

Tricky/Scenario-Based Questions

- If a class has only static methods, should it be instantiated? How can we enforce this?
- If we create two objects of a class, how many copies of static and instance variables exist?
- What happens if you initialize a static variable inside the constructor?
- How does Java handle static final variables?
- What happens if a static block throws an exception?